

Lesson 1: The Core Motivation & Anatomy of an Address Space

Slide Range: Slide 1 to Slide 5[cite: 1]

The Big Picture: Why Are We Here? (Slide 1 & 2)

Imagine you buy an 8 GB RAM stick, snap it onto your motherboard, and turn on your computer[cite: 1]. You launch Chrome, Spotify, and a code editor all at once.

At the hardware level, there is only **one** set of physical memory pins wired to that motherboard.

Now ask yourself:

- How does Chrome know where its data lives without accidentally overwriting Spotify's memory[cite: 1]?
- What if Chrome has a bug, goes wild, and tries to write to byte `0x0000 1234` [cite: 1]? Does it wipe out your Spotify song playback[cite: 1]?
- Worse, when the programmer at Google compiled Chrome, how could they possibly know in advance what other applications you would be running, or which physical RAM slots would be free when you clicked the icon[cite: 1]?

The answer is: **They couldn't**[cite: 1].

If software had to talk directly to physical RAM, modern operating systems would be impossible to run safely.

This brings us to **Memory Virtualization**[cite: 1]:

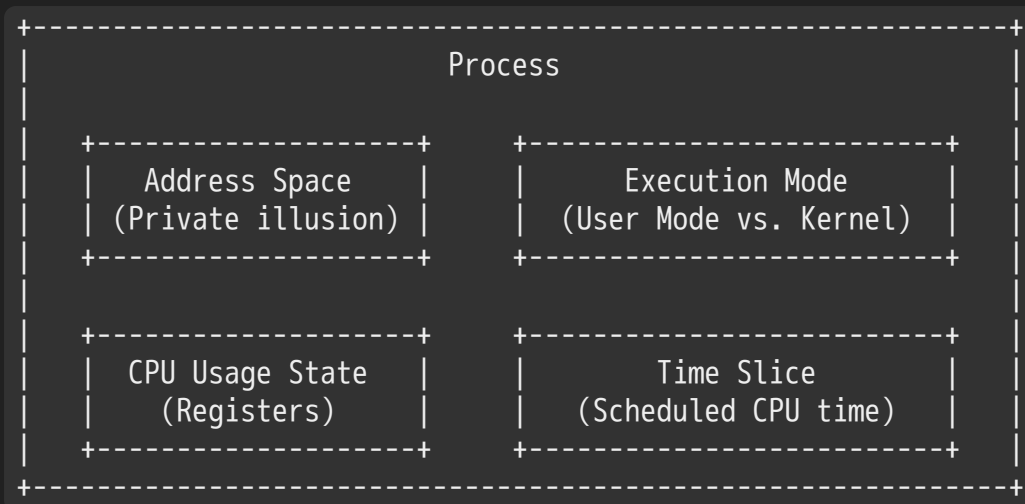
The Core Concept: The Operating System (OS) sits between your program and the physical hardware, actively lying to every single process[cite: 1].

The OS tells Chrome: *"You have the entire memory space from 0 to max all to yourself*[cite: 1]. *Don't worry about anyone else."* It tells the exact same lie to Spotify, and the exact same lie to your code editor[cite: 1].

Every process lives inside its own private illusion called a **Virtual Address Space**[cite: 1].

The Process as a Unit of Isolation (Slide 3)

Because of this illusion, the OS treats a **Process** as the fundamental **Unit of Isolation**[cite: 1]:

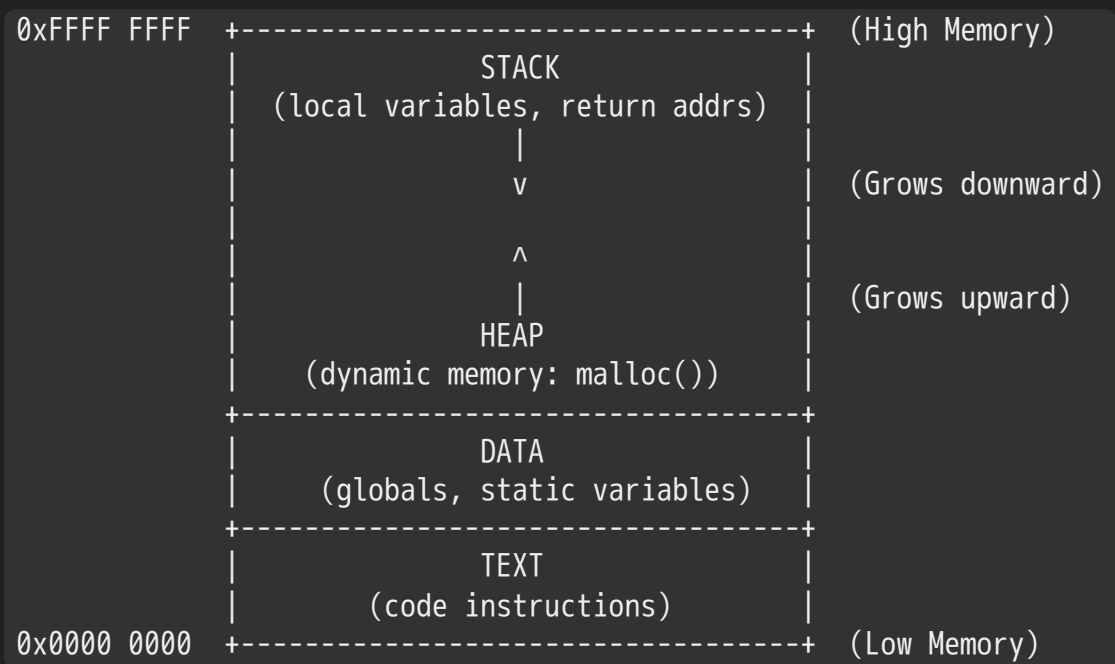


1. **Address Space:** The isolated sandbox of memory addresses this program can see[cite: 1].
2. **Execution Mode:** A hardware CPU flag[cite: 1]. Your program runs in **User Mode** (restricted mode) [cite: 1]. It cannot touch physical devices or raw memory directly; it must ask the OS for permission[cite: 1].
3. **Time Slice & State:** The OS gives the process a few milliseconds of CPU time, pauses it, saves its register states, and switches to another process[cite: 1].

Isolation guarantees that if Process A crashes, segfaults, or executes garbage instructions, Process B continues running completely unaffected[cite: 1].

Inside the Sandbox: The Anatomy of an Address Space (Slide 4)

When your program is running inside its virtual world, how is its memory arranged? The OS lays it out into four functional segments[cite: 1]:



1. TEXT (Code Segment):

- Located at the bottom (starting near `0x0000 0000`) [cite: 1].

- This is where your compiled assembly instructions (`mov` , `add` , `jmp`) live[cite: 1].
- Marked **Read-Only** by the OS so the program cannot accidentally overwrite its own code instructions during execution.

2. DATA Segment:

- Sits directly above the text segment[cite: 1].
- Holds **global variables** and **static variables**[cite: 1]. These are variables that must survive for the entire lifetime of the program, from start to termination[cite: 1].

3. HEAP Segment:

- This is **Dynamic Memory**[cite: 1].
- When you write `malloc(100)` in C or `new Object()` in C++/Java, you are requesting bytes on the fly at runtime[cite: 1].
- The heap expands **downward** toward higher memory addresses as you allocate more space[cite: 1].

4. STACK Segment:

- Sits at the top of user space and expands **upward** toward lower memory addresses[cite: 1].
- Holds **local variables** declared inside functions, **function parameters**, and **return addresses** (so the CPU knows which line of code to jump back to when a function returns)[cite: 1].

Why do Stack and Heap grow toward each other?

Because nobody knows in advance how much stack or heap a program will use[cite: 1]. If you fixed their sizes side-by-side, one might run out of space even if the other had plenty left over. By letting them grow toward each other from opposite ends, they share the free space in the middle dynamically[cite: 1].

How a File on Disk Becomes a Living Process (Slide 5)

Let's bridge the gap between source code and memory[cite: 1].

Suppose you write this simple program[cite: 1]:

```
c
#include <stdio.h>
int main() {
    printf("Hello");
    return 0;
}
```

What actually happens step-by-step to run this[cite: 1]?

```
[ hello.c (Source Code) ]
|
v (Compiler: gcc)
[ hello.o / a.out (Binary file on Disk) ]
|
v (OS Loader reads ELF & Magic Number)
[ Virtual Address Space Allocated in RAM ]
|
v (Set PC / EIP to main())
[ Process Begins Executing Instructions ]
```

1. From Source to Binary File

When you write `hello.c`, it is just a plain ASCII text file[cite: 1]. When you run `gcc hello.c`, the compiler translates those text statements into raw machine code (hexadecimal opcodes) packaged into an executable file on your hard disk[cite: 1].

2. How the OS Recognizes an Executable (The Magic Number)

Windows relies on file extensions like `.exe`. Linux and teaching operating systems like xv6 do not rely on extensions[cite: 1]. If you rename your binary to `myprogram.txt` or remove the extension completely, the OS can still execute it[cite: 1].

How? The OS inspects the very first 4 bytes of the binary file, known as the **Magic Number**[cite: 1].

- In Linux, those bytes are: `0x7F`, followed by `'E'`, `'L'`, `'F'` (hex values `45 4c 46`), representing the **ELF (Executable and Linkable Format)**[cite: 1].
- When the OS loader reads `0x7f 45 4c 46`, it knows: *"This is a valid compiled binary. I know how to load this."*[cite: 1]

3. Building the Address Space in RAM

The OS reads the header metadata inside the ELF binary to see how much code and static data exist[cite: 1]. Then it performs the setup[cite: 1]:

- It reserves memory for the process[cite: 1].
- It copies the compiled machine bytes from the disk file into the **Code (Text)** segment[cite: 1].
- It sets up the **Data** segment[cite: 1].
- It allocates initial space for the **Stack** and the **Heap**[cite: 1].

4. Setting the Program Counter (PC / EIP)

The CPU cannot execute code without an instruction pointer. The OS sets the CPU's **Program Counter (PC or EIP register)** to point directly to the first instruction of your program's entry point[cite: 1]. The CPU drops out of kernel mode into user mode, reads the instruction pointed to by the PC, and execution begins[cite: 1].

5. Why Disassembly (`objdump -d`) Is Huge

If you run `objdump -d a.out` on your tiny `printf("Hello")` binary, you get thousands of lines of assembly code[cite: 1].

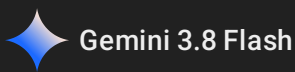
- You only wrote 4 lines of C code, so why is the binary so large[cite: 1]?
- Because your program called `printf()` [cite: 1]. The implementation of `printf()` lives in the standard C library (`libc.a` or dynamic libraries)[cite: 1]. The linker brings in all the pre-compiled library routines and runtime initialization code into your address space so that `printf` can actually format text and communicate with the OS[cite: 1]. Those library functions are pre-compiled once; they do not need to be written or recompiled from scratch[cite: 1].

Let me know if this pace and depth work for you, and we will move to **Lesson 2 (Slides 6 to 10)** to look at how the Kernel maps itself into this space and how the CPU switches between User and Kernel stacks[cite: 1].

2026-09-03 12:56:48

You 

like this.. next



Lesson 2: The Kernel-User Split, Process Internals & Dual Stacks

Slide Range: Slide 6 to Slide 10[cite: 1]

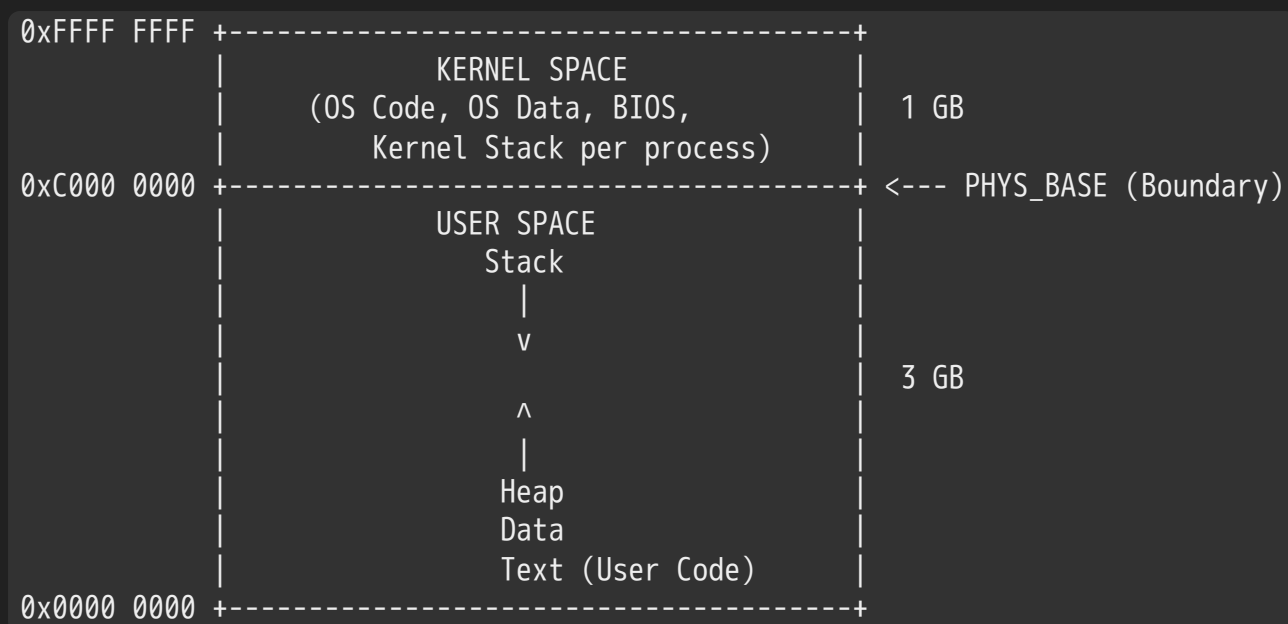
The Big Question: Where Does the Kernel Live? (Slide 6)

In Lesson 1, we learned that every process gets an illusion of having its own massive virtual address space[cite: 1]. On a 32-bit system, that space spans from `0x0000 0000` to `0xFFFF FFFF` —exactly 4 GB of addressable space[cite: 1].

Now, think about this like an OS architect:

- If the user process thinks it owns all 4 GB, where does the Operating System code sit[cite: 1]?
- When your program calls `printf()` and needs the OS to print characters on the screen, does the CPU throw away the process's memory to bring in the kernel[cite: 1]?

That would be disastrously slow. Instead, the designers split every process's 4 GB virtual address space into two distinct regions[cite: 1]:



- **User Space (Bottom 3 GB: 0x0000 0000 to 0xC000 0000):** This is where your code, your variables, your heap allocations, and your linked libraries (like `libc.a`) live[cite: 1]. Your program has full read and write access here[cite: 1].
- **Kernel Space (Top 1 GB: 0xC000 0000 to 0xFFFF FFFF):** The OS kernel maps *itself* directly into the top of your address space[cite: 1]!
- **The Catch:** Even though the kernel is mapped right above your stack, the CPU hardware blocks access while your code runs in **User Mode**[cite: 1]. If your code tries to read or write an address above `0xC000 0000` directly, the hardware CPU throws a segmentation fault / exception immediately[cite: 1].

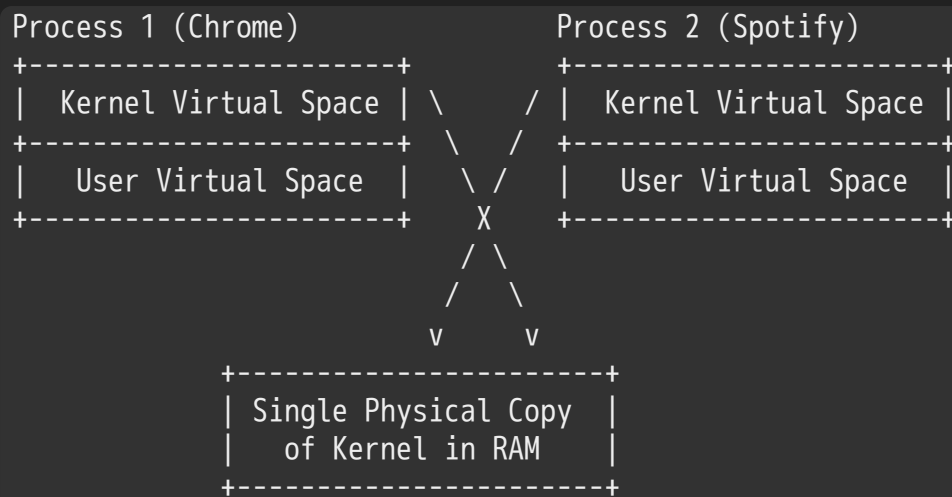
Are There Multiple Copies of the Kernel? (Slide 7)

Suppose you have 10 programs running simultaneously: Chrome, Spotify, a compiler, and 7 terminal windows[cite: 1].

Does the computer copy the OS kernel 10 separate times into physical RAM[cite: 1]?

No, absolutely not[cite: 1].

Look at what happens behind the scenes[cite: 1]:

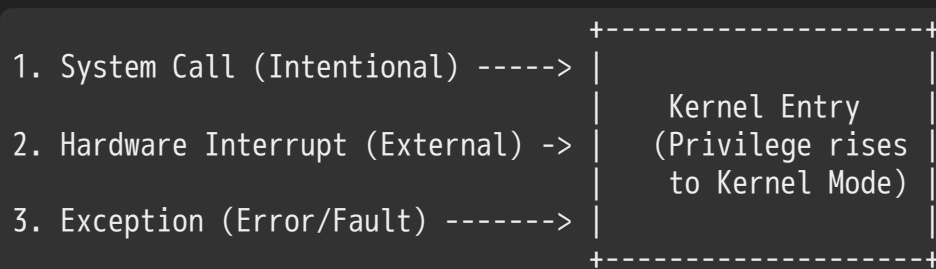


- There is only **one single physical copy** of the OS kernel and its core routines sitting in your physical RAM chips[cite: 1].
- Every process's address translation table points its top 1 GB virtual kernel addresses right back to that exact same physical location in RAM[cite: 1].
- **However:** Each process maintains its own separate **Kernel Stack** so that when the kernel does work on behalf of a specific process, it has its own private workspace[cite: 1].

Why Would a User Process Jump into the Kernel? (Slide 7)

Your program is happily executing its own code in User Mode[cite: 1]. What causes it to suddenly hand control over to the Kernel[cite: 1]?

There are exactly three ways this happens[cite: 1]:



1. System Call (Intentional):

Your program asks the OS to do something it does not have the hardware permission to do itself (e.g., read a file from the SSD, send a packet through the Wi-Fi card, or allocate more heap memory)[cite: 1]. You voluntarily trigger a software trap instruction to request OS assistance[cite: 1].

2. Hardware Interrupt (External & Asynchronous):

Your program did nothing wrong[cite: 1]. But while it was computing a loop, you pressed a key on the keyboard, or the network card received a packet, or the CPU timer chip ticked[cite: 1]. The CPU hardware forces a pause on your code and immediately jumps to the kernel's interrupt handler to process the hardware device[cite: 1].

3. Exception (Internal Error):

Your program executed an illegal operation—such as dividing an integer by zero, executing an invalid opcode, or trying to access memory outside its boundaries[cite: 1]. The CPU halts your execution and jumps into the kernel exception handler, which will typically terminate your process (e.g., "Segmentation Fault")[cite: 1].

How the OS Tracks You: `struct thread` (Slide 8 & 9)

Inside the operating system's source code (such as the Pintos teaching OS), how does the kernel actually represent and track a running program[cite: 1]?

It uses a Process Control Block (PCB), defined in Pintos as `struct thread` [cite: 1]:

```
c

struct thread {
    tid_t tid;                /* Thread identifier (like a PID) */
    enum thread_status status; /* Current state: RUNNING, READY, BLOCKED,
DYING */
    char name[16];            /* Name of process (for debugging) */
    uint8_t *stack;           /* Pointer to saved KERNEL stack */
    int priority;              /* Scheduling priority */
    struct list_elem allelem;  /* Global list tracking all threads */
    struct list_elem elem;     /* Ready-queue tracking element */

#ifdef USERPROG
    uint32_t *pagedir;        /* Pointer to this process's Page Table
Directory */
#endif

    unsigned magic;           /* Stack overflow detection canary */
};
```

Let's break down the most critical fields[cite: 1]:

- **`pagedir` (The Page Directory):**

This is the pointer to the process's page table[cite: 1]. This table holds the translation maps converting this specific process's virtual addresses into physical addresses in RAM[cite: 1].

- **`stack` (The Kernel Stack Pointer):**

When this process is paused so another process can run, the CPU registers are saved onto this thread's private kernel stack, and this pointer saves where that stack ended[cite: 1].

- **`status` (The Lifecycle State):**

- `THREAD_RUNNING` : Currently running on the physical CPU core[cite: 1].

- **THREAD_READY** : Ready to run, waiting in the scheduler's queue for its turn[cite: 1].
- **THREAD_BLOCKED** : Paused, waiting for an event (e.g., waiting for the disk to finish reading data)[cite: 1].
- **THREAD_DYING** : The process is exiting and about to be destroyed[cite: 1].
- **magic** :
A special random number placed at the very edge of the kernel stack[cite: 1]. If the stack overflows and grows too big, it overwrites this number[cite: 1]. The OS checks this value to catch stack overflow crashes before they corrupt memory[cite: 1].

Process vs. Thread: A Crucial Distinction (Slide 9)

People often confuse processes and threads, but their roles are clearly separated[cite: 1]:

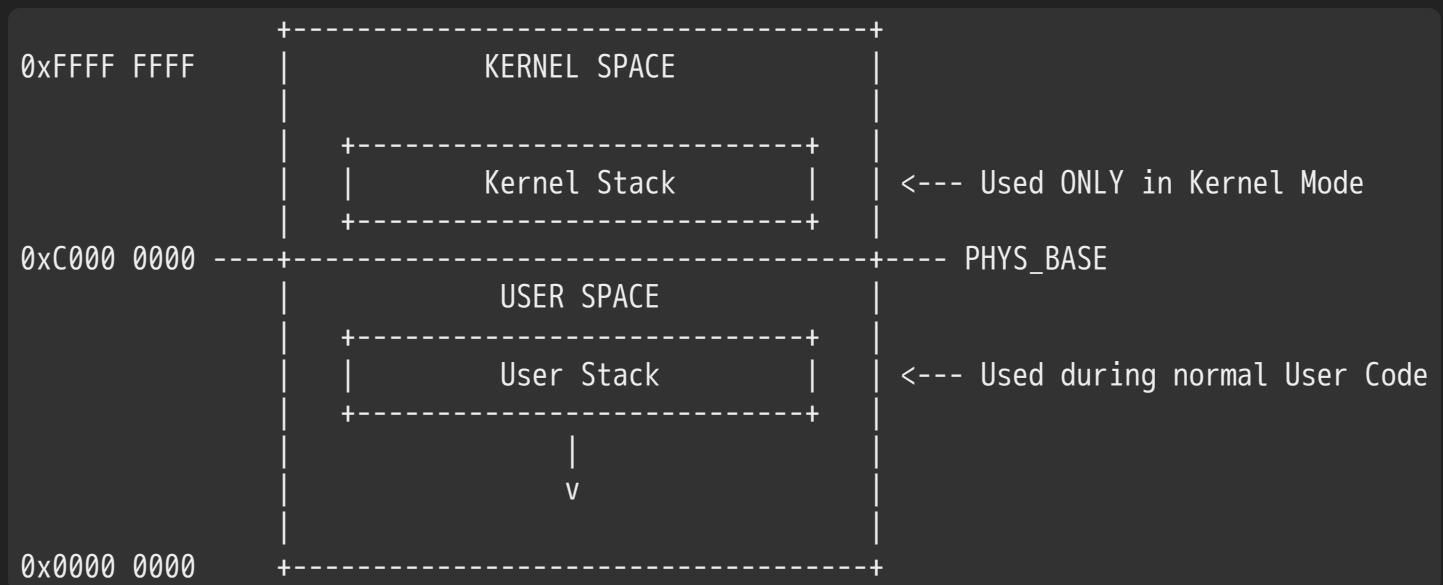
Entity	Role	What It Owns
Process	Unit of Isolation[cite: 1]	Page Table (pagedir) [cite: 1]. It isolates memory so no other process can view or modify its space[cite: 1].
Thread	Unit of Execution[cite: 1]	CPU Registers & Stacks[cite: 1]. It tracks where the instruction pointer is (EIP / PC) and keeps local function variables[cite: 1].

If you have a multi-threaded application (e.g., Chrome with multiple tabs), all those threads share the **same pagedir** (they see the exact same heap, code, and data)[cite: 1]. However, every individual thread has its **own** private stack and saved registers[cite: 1]!

The Two Stacks: User Stack vs. Kernel Stack (Slide 10)

Here is a critical concept: **Every process has two separate stacks**[cite: 1].

1. The **User Stack** (lives down in User Space)[cite: 1].
2. The **Kernel Stack** (lives up in Kernel Space)[cite: 1].



Why can't the Kernel just use the User's Stack?

Imagine your program is running, and it sets its stack pointer (`%esp`) to an invalid address or to a completely full stack on purpose. If the OS kernel tried to use the user stack during a system call, the kernel would instantly crash or expose sensitive kernel operations to the user space!

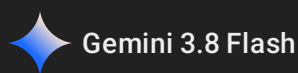
To guarantee security and stability, user code and kernel code are never allowed to share the same stack[cite: 1].

What Happens During a System Call Transition?

When your program calls a system call (like `write()`):

- Switch Privilege Level:** The CPU hardware switches privilege mode from User Mode (Ring 3) to Kernel Mode (Ring 0)[cite: 1].
- Switch the Stack:** The CPU immediately switches its active stack pointer register (`%esp`) from the **User Stack** to the process's private **Kernel Stack**[cite: 1].
- Save State:** The CPU hardware pushes the user's return address and user stack pointer onto this kernel stack so it remembers where to return later[cite: 1].
- Execute & Return:** The kernel runs the requested service, switches the stack back to the User Stack, lowers privilege back to User Mode, and resumes user code execution right where it left off[cite: 1].

Let me know if you are ready to move to **Lesson 3 (Slides 11 to 14)**, where we will see live C code proving these virtual memory addresses, explore the BSS segment optimization, and understand why the stack randomizes its address to defeat hackers (ASLR)[cite: 1]!

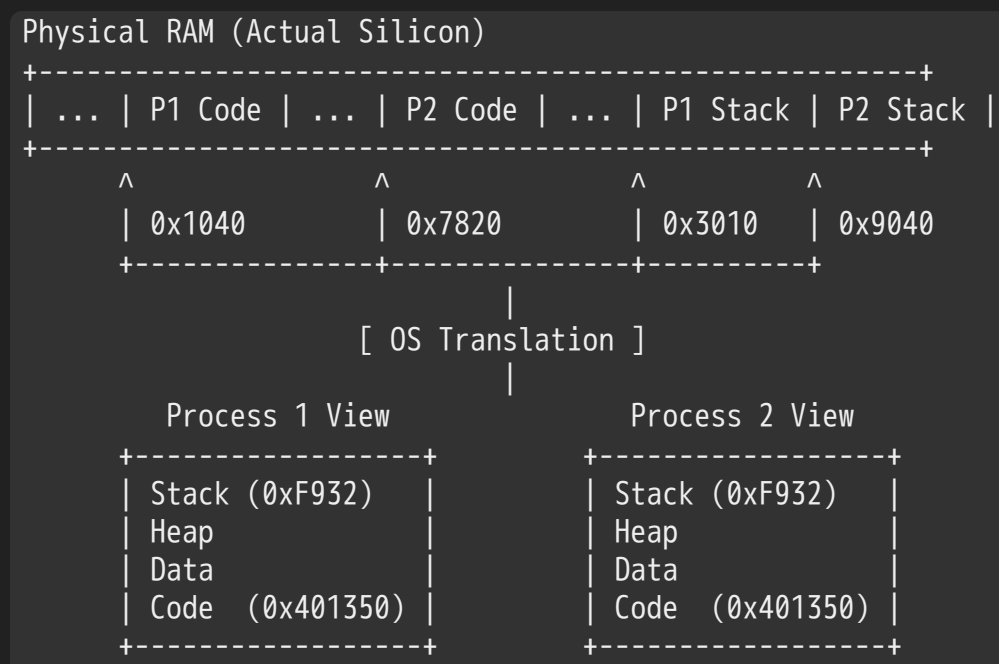


Lesson 3: Live Code Experiments, the BSS Segment, and Memory Randomization (ASLR)

Slide Range: Slide 11 to Slide 14[cite: 1]

The Core Illusion in Action (Slide 11)

To build intuition, consider two independent programs running simultaneously:



Inside their respective virtual spaces, both Process 1 and Process 2 have an integer variable `x` located at logical address `0xF932` [cite: 1].

- Does variable `x` physically exist at wire `0xF932` on the RAM stick[cite: 1]? **No**[cite: 1].
- Both programs reference virtual address `0xF932`, but the OS translates Process 1's virtual address to physical address `0x3010`, and Process 2's virtual address to physical address `0x9040` [cite: 1].
- This setup provides **isolation** and **ease of programming**: software authors do not have to coordinate or track where their programs are placed in physical RAM[cite: 1].

Proving the Virtual Address Space with Real C Code (Slide 12)

We can verify how the compiler and OS assign these addresses using simple C code examples[cite: 1].

Experiment 1: The Code Segment

C

```
#include <stdio.h>

void func() {}

int main() {
    printf("code segment address: %p\n", func);
    while (1);
    return 0;
}
```

- **Observation:** Compile this program, open two terminal windows, and run it twice at the same time[cite: 1].
- **Output:** Both instances print `0x00401350` [cite: 1].
- **Explanation:** Two distinct programs cannot occupy the exact same physical transistors in RAM simultaneously. The address `0x00401350` is a **virtual address**[cite: 1]. Each process has its own page table mapping that virtual `0x00401350` to a different physical address frame[cite: 1].

Experiment 2: Initialized Global Variables

C

```
#include <stdio.h>

void func() {}
int globy = 5;

int main() {
    printf("data segment          : %p\n", &globy);
    printf("code segment address: %p\n", func);
    while (1);
    return 0;
}
```

- **Output:**
 - `code segment address: 0x00401350` [cite: 1]
 - `data segment : 0x00402000` [cite: 1]
- **Explanation:** The initialized global variable `globy` resides in the **Data Segment**, placed right above the executable Code segment in memory[cite: 1].

Experiment 3: Uninitialized Data and the BSS Optimization

```

C
#include <stdio.h>

void func() {}
int globy = 5;
int globalUninitialized;

int main() {
    printf("BSS segment      : %p\n", &globalUninitialized);
    printf("data segment     : %p\n", &globy);
    printf("code segment address: %p\n", func);
    while (1);
    return 0;
}

```

- **Output:**

- code segment address: 0x00401350 [cite: 1]
- data segment : 0x00402000 [cite: 1]
- BSS segment : 0x00405008 [cite: 1]

- **Why does BSS (Block Started by Symbol) exist?**

- If you define a global array like `int arr[1000000];` without initial values, writing 4 MB of zeros directly into your compiled executable on the SSD would waste disk space.
- Instead, the compiler tags it as **BSS**[cite: 1]. The binary on disk only records the metadata: *"I need 4 MB of uninitialized global memory."*
- When the program launches, the OS zeroes out that memory range in RAM in bulk, saving storage space and speeding up program loading[cite: 1].

Experiment 4: Stack Garbage and the Stack Pointer

```

C
#include <stdio.h>
#include <stdlib.h>

void func() {}
int globy = 5;
int globalUninitialized;

int main() {
    int x;
    printf("stack segment      : %p\n", &x);
    printf("Heap segment         : %p\n", malloc(1));
}

```

```

printf("BSS segment      : %p\n", &globalUninitialized);
printf("data segment     : %p\n", &globy);
printf("code segment address: %p\n", func);
while (1);
return 0;
}

```

- **Output Hierarchy:**

- Stack : 0x0060FFFC (High memory)[cite: 1]
- Heap : 0x009D0DA0 (Dynamic memory allocated above static data)[cite: 1]
- BSS : 0x00405008 [cite: 1]
- Data : 0x00402000 [cite: 1]
- Code : 0x00401350 (Low memory)[cite: 1]

- **Why Local Variables Have "Garbage Values":**

When a function exits, the CPU does not clear its memory to zero. It merely moves the stack pointer register (`%esp`) backward[cite: 1]. The previous bits remain in RAM[cite: 1]. When a new function allocates stack space without assigning initial values, it reads whatever bit patterns were left behind by earlier functions[cite: 1].

The Evolution of Memory Management (Slide 13)

To understand modern memory systems, it helps to examine earlier designs:

1. Uniprogramming (Early Systems)

- Early systems loaded only **one single process** into memory at a time alongside the OS[cite: 1].
- **Drawback:** Inefficiency[cite: 1]. If the program paused for I/O (like reading from disk or waiting for user input), the CPU remained idle[cite: 1].

2. Multiprogramming and Time Sharing

- To optimize CPU usage, multiple processes were kept in RAM at once[cite: 1]. When Process A paused for I/O, the CPU switched to Process B[cite: 1].
- **Fixed Partition Limitations:**
 - **Memory Waste & Inflexibility:** If memory is divided into fixed 64 KB slots, a process requiring more than 64 KB cannot run at all[cite: 1].
 - **Lack of Protection:** Because processes sat side-by-side in physical memory without virtual translation, a bug in Process B could write directly into Process A's memory or crash the OS[cite: 1].

Address Space Layout Randomization (ASLR) (Slide 14)

Under standard static assumptions, running the same binary twice should produce identical memory addresses[cite: 1]. However, on modern systems, executing the program repeatedly yields different Stack and Heap addresses each time[cite: 1].

This security feature is called **Address Space Layout Randomization (ASLR)**[cite: 1].

The Vulnerability: Buffer Overflow

Consider a function with an unchecked local buffer[cite: 1]:

```
c
void vulnerable_function() {
    char p[2];
    strcpy(p, "attacker_controlled_malicious_string");
}
```

Stack Growth Direction (Downward towards lower addresses)

[Local Buffer `p`] --> [Saved Frame Pointer] --> [Return Address (EIP)]

- Because `p` is allocated on the stack, writing beyond its 2-byte boundary overflows into adjacent stack memory[cite: 1].
- The **return address** (which tells the CPU where to resume execution after the function finishes) is stored on the stack directly after the local variables[cite: 1].
- **Without ASLR:** An attacker can predict the exact address of their injected code in memory and overwrite the return address with that pointer[cite: 1]. When the function returns, it jumps straight into the exploit code[cite: 1].
- **With ASLR:** The OS introduces random offsets to the base of the Stack and Heap on every program run[cite: 1]. Because the memory locations shift dynamically, attackers cannot reliably target return addresses or jump locations[cite: 1].

Let me know if you are ready to conclude Chapter 13 with **Lesson 4 (Slides 15 to 19)**, covering Base & Bound registers, hardware-level relocation, and dynamic heap allocation alignment[cite: 1].

2026-09-03 12:57:31

You 

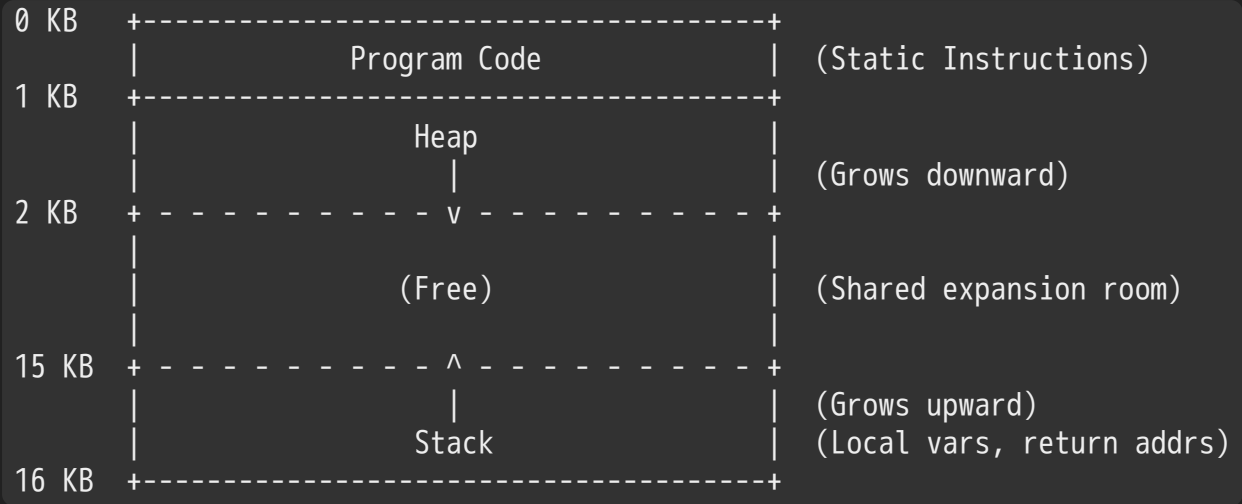
yes next

Lesson 4: Hardware Relocation (Base & Bound), 64-Bit Memory Maps, and Heap Alignment

Slide Range: Slide 15 to Slide 19[cite: 1]

The Elastic Core: Stack vs. Heap Mechanics (Slide 15)

In earlier lessons, we saw that the address space contains code, dynamic data, and local variables[cite: 1]. Slide 15 zooms in on a simplified **16 KB address space** to show how memory expansion works dynamically[cite: 1]:

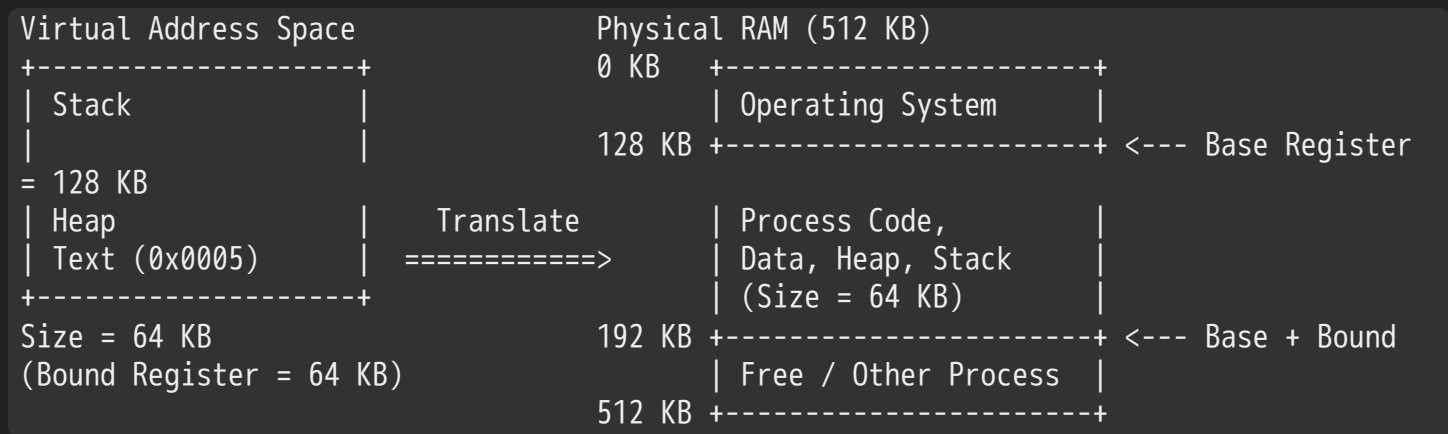


- **Program Code (0 KB – 1 KB):** Holds machine instructions[cite: 1]. Its size is fixed at compile time, so it stays pinned at the base[cite: 1].
- **Heap (1 KB – 2 KB):** Handles dynamic runtime requests via `malloc()` in C or `new` in object-oriented languages[cite: 1]. It expands downward toward higher address values[cite: 1].
- **Stack (15 KB – 16 KB):** Holds local variables, routine arguments, and return addresses[cite: 1]. It expands upward toward lower address values[cite: 1].
- **The Free Gap in the Middle:** Both dynamic regions grow into the unallocated space between them[cite: 1]. The program only runs out of memory if the heap and stack collide[cite: 1].

The Hardware Shield: Base and Bound Registers (Slide 16)

When multiple processes share a single physical RAM chip, how does the CPU stop one process from corrupting another's memory[cite: 1]?

To solve this, the CPU hardware provides two dedicated registers: the **Base Register** and the **Bound (or Limit) Register**[cite: 1].



1. The Translation Formula

Whenever a program accesses a virtual memory address, the CPU hardware translates it on the fly before sending it to the RAM pins[cite: 1]:

$$\text{Physical Address} = \text{Virtual Address} + \text{Base}$$

If Process A has a base of 128 KB and tries to read its own instruction at virtual address 0x0005, the CPU performs the addition:

$$128 \text{ KB} + 5 = \text{Physical Byte } 131077$$

The running program does not need to know where it is physically placed[cite: 1]; it accesses memory as if it starts at address zero[cite: 1].

2. The Protection Check

Before sending the calculated address to RAM, the CPU performs a hardware bounds check[cite: 1]:

$$0 \leq \text{Virtual Address} < \text{Bound}$$

- If the program tries to access a virtual address beyond its allocated size (for example, address 70 KB when its bound is 64 KB), the CPU halts execution immediately[cite: 1].
- It triggers an **exception/fault**, transferring control to the OS kernel, which terminates the offending process with a segmentation fault[cite: 1].

3. Privileged Protection

What stops a process from executing `load $base, 0x00` in assembly to manipulate its own Base register[cite: 1]?

- Modifying the Base and Bound registers is a **privileged instruction**[cite: 1].
- The CPU verifies the current execution mode[cite: 1]. In **User Mode**, running this instruction throws a privilege exception and halts the process[cite: 1]. Only the OS running in **Kernel Mode** can configure these registers during a context switch[cite: 1].

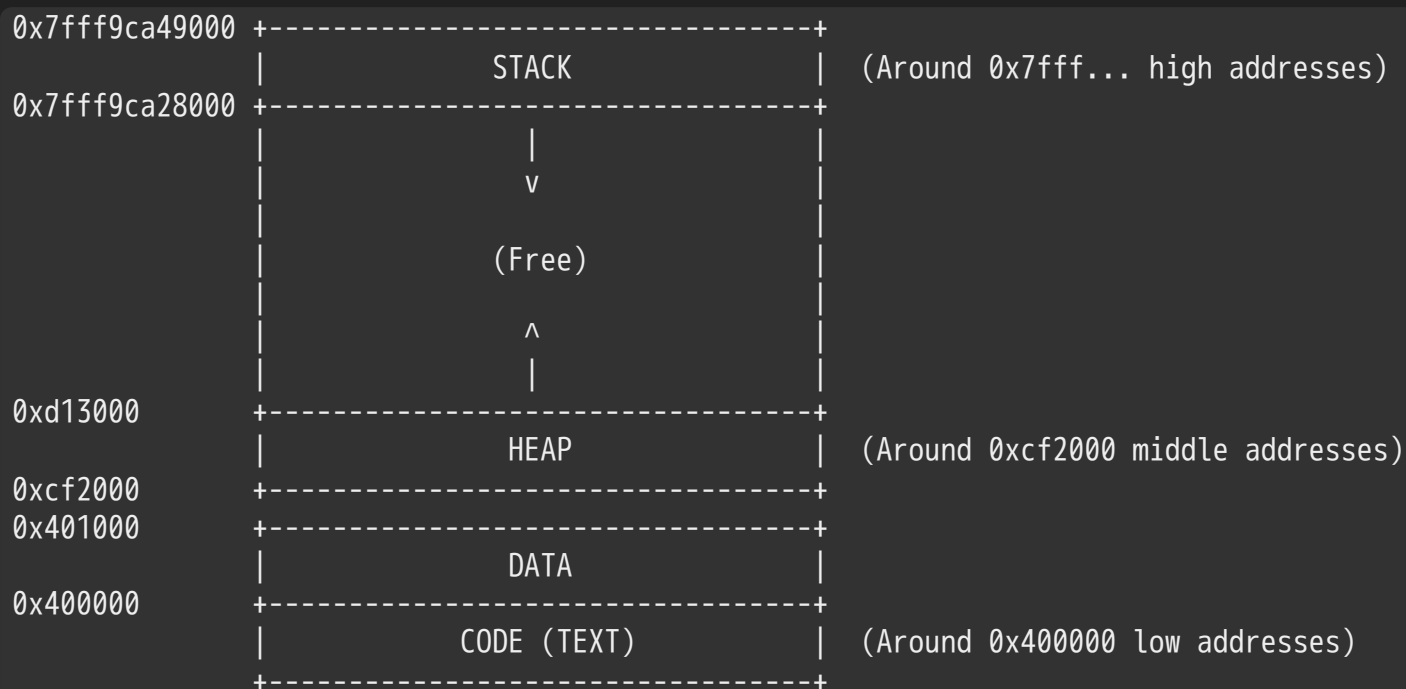
Slide 17 tests these concepts with a 64-bit Linux program[cite: 1]:

```
c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    printf("location of code : %p\n", (void *) main);
    printf("location of heap : %p\n", (void *) malloc(1));
    int x = 3;
    printf("location of stack: %p\n", (void *) &x);
    return x;
}
```

Output on a 64-Bit Linux Machine

- location of code : 0x40057d [cite: 1]
- location of heap : 0xcf2010 [cite: 1]
- location of stack: 0x7fff9ca45fcc [cite: 1]



The output confirms the theoretical layout in practice[cite: 1]:

- Code resides in low memory (0x40057d) [cite: 1].
- The heap lives above the static binary footprint (0xcf2010) [cite: 1].
- The stack begins at the high end of user space (0x7fff9ca45fcc) [cite: 1].

Live Verification: Stack Offsets and Heap Alignment (Slides 18 & 19)

The final program inspects each memory region simultaneously[cite: 1]:

```
c

#include <stdio.h>
#include <stdlib.h>

int InitializedGlobal[1024] = {0,};
int UninitGlobal[1024];

int main() {
    int localVar1;
    int localVar2;
    int *dynamicLocalVar1;
    int *dynamicLocalVar2;

    dynamicLocalVar1 = malloc(sizeof(int));
    dynamicLocalVar2 = malloc(sizeof(int));

    printf("code           : 0x%x\n", main);
    printf("Data           : 0x%x\n", &InitializedGlobal);
    printf("BSS (Uninit Data)   : 0x%x\n", &UninitGlobal);
    printf("stack localVar1     : 0x%x\n", &localVar1);
    printf("stack localVar2     : 0x%x\n", &localVar2);
    printf("heap dynamicLocalVar1: 0x%x\n", dynamicLocalVar1);
    printf("heap dynamicLocalVar2: 0x%x\n", dynamicLocalVar2);

    return 0;
}
```

Output & Layout Analysis (Slide 19)

Output Line[cite: 1]	Printed Address[cite: 1]	Memory Segment[cite: 1]
code [cite: 1]	0xdffde0 [cite: 1]	Code / Text (Lowest)[cite: 1]
Data [cite: 1]	0xe01020 [cite: 1]	Initialized Data[cite: 1]
BSS (Uninit Data) [cite: 1]	0xe02020 [cite: 1]	BSS (Zero-filled Globals)[cite: 1]
heap dynamicLocalVar1 [cite: 1]	0xc4c01700 [cite: 1]	Heap (First Chunk)[cite: 1]
heap dynamicLocalVar2 [cite: 1]	0xc4c01710 [cite: 1]	Heap (Second Chunk)[cite: 1]
stack localVar2 [cite: 1]	0xeeee00a84 [cite: 1]	Stack (Allocated 2nd)[cite: 1]
stack localVar1 [cite: 1]	0xeeee00a88 [cite: 1]	Stack (Allocated 1st)[cite: 1]

1. Demonstrating Stack Growth

- `localVar1` received address `0xee00a88` [cite: 1].
- `localVar2` received address `0xee00a84` [cite: 1].
- The address decreased by 4 bytes:

$$0xee00a88 - 0xee00a84 = 4 \text{ bytes}$$

Because each `int` occupies 4 bytes, declaring `localVar2` moved the stack pointer downward toward lower addresses[cite: 1].

2. The 16-Byte Heap Allocation Unit

- `dynamicLocalVar1` sits at `0xc4c01700` [cite: 1].
- `dynamicLocalVar2` sits at `0xc4c01710` [cite: 1].
- The difference between the two pointers is:

$$0xc4c01710 - 0xc4c01700 = 16 \text{ bytes (0x10)}$$

Even though the code only requested `sizeof(int)` (4 bytes), the heap pointers differ by 16 bytes[cite: 1].

As annotated on Slide 19: "**Minimum heap allocation unit: 16 Byte**"[cite: 1]. The memory allocator (`malloc`) rounds allocations up to 8-byte or 16-byte alignment boundaries for memory bus efficiency and keeps internal metadata headers (such as chunk size and status flags) alongside each allocation[cite: 1].

This covers all slides, code walkthroughs, and annotations for Chapter 13: Address Space[cite: 1]. Which chapter are you tackling next?